

PROGRAMMATION FONCTIONNELLE 2 : TD10 PRÉDICATS INDUCTIFS, ENCORE

avril 2026 — Pierre Rousselin

On rappelle qu'on a défini sur `nat` le prédicat inductif `le` et la fonction booléenne `leb` de la façon suivante :

```
Inductive le (n : nat) : nat -> Prop :=
  | le_n : le n n | le_S : forall m : nat, le n m -> le n S m.
Notation "n <= m" := (le n m).
Fixpoint leb n m :=
  match n with
  | 0 => true
  | S n' => match m with
    | 0 => false
    | S m' => leb n' m'
  end
end.
Notation "n <=? m" := (leb n m).
```

Par ailleurs, on a les lemmes de réflexion :

```
Lemma le_leb (n m : nat) : n <= m -> n <=? m = true. Proof. (* ... *) Qed.
Lemma leb_le (n m : nat) : n <=? m = true -> n <= m. Proof. (* ... *) Qed.
```

Le prédicat correspondant intuitivement à « être une liste triée d'entiers naturels » est :

```
Inductive sorted : list nat -> Prop :=
  | nil_sorted : sorted []
  | one_sorted (x : nat) : sorted [x]
  | cons_sorted (h1 h2 : nat) (t : list nat) :
    sorted (h2 :: t) -> h1 <= h2 -> sorted (h1 :: h2 :: t).
```

Exercice 1 :

1. Réécrire la définition de `sorted` sous la forme de règles d'inférences.
2. Donner l'arbre de dérivation pour prouver `sorted [10; 42; 93]`.
3. Donner la définition en Rocq de la fonction

```
(** retourne la liste [start; S (start); ... (start + len - 1)] *)
Fixpoint list_of_nats (start len : nat) := (* à compléter *)
```

4. Donner la preuve papier et/ou Rocq du lemme suivant :

```
Lemma sorted_list_of_nat (start len : nat) : sorted (list_of_nats start len).
```

Faire attention à la généralité de l'hypothèse d'induction (il est instructif de se lancer dans une induction non généralisée et de voir à quel endroit ça coince).

--- * ---

Exercice 2 : Permutation

Le prédicat inductif correspondant intuitivement à « deux listes sont permutation l'une de l'autre » est défini de la façon suivante :

```
Inductive Permutation {A : Type} : list A -> list A -> Prop :=
  | perm_nil : Permutation [] []
  | perm_skip (x : A) (l l' : list A) :
    Permutation l l' -> Permutation (x :: l) (x :: l')
  | perm_swap (x y : A) (l : list A) :
    Permutation (y :: x :: l) (x :: y :: l)
  | perm_trans (l l' l'' : list A) :
    Permutation l l' -> Permutation l' l'' -> Permutation l l''.
```

1. Donner la preuve en Rocq de

Lemma `Permutation_exple1` : `Permutation [1; 42; 93] [1; 93; 42]`.

2. Idem pour

Lemma `Permutation_exple2` : `Permutation [42; 93; 2; 1] [93; 42; 1; 2]`.

3. Prouver, en Rocq et/ou sur papier, par induction sur la preuve de `Permutation l1 l2` :

Lemma `Permutation_length` {`A : Type`} (`l1 l2 : list A`) :
`Permutation l1 l2 -> length l1 = length l2`.

--- * ---

Exercice 3 : Preuve par cas « trop destructive » sur `sorted`

Terminer :

— le nommage dans les cas de `destruct`,

— la preuve en Rocq, en remplaçant chaque fois `l` par la liste correspondante

dans la preuve suivante :

Lemma `sorted_cons` (`h : nat`) (`t : list nat`) :
`sorted (h :: t) -> sorted t`.

Proof.

`remember (h :: t) as l eqn: def_1.`
`intros H; destruct H as [...]`.

La tactique

`remember (h :: t) as l eqn: def_1.`

introduit dans le contexte l'équation

`def_1 : l = h :: t`

et change toutes les autres occurrences `h :: t` du contexte et du but en `l`.

--- * ---

Exercice 4 : Réflexion booléenne pour `sorted`

1. Donner la définition en Rocq de la fonction `sortedb` : `list nat -> bool` qui vaut `true` si et seulement si son argument est une liste triée dans l'ordre croissant.

2. Quelle est la preuve commune des deux lemmes suivants?

Lemma `sortedb_cons_cons_leb` (`h h' : nat`) (`l : list nat`) :
`h <= h' = true -> sortedb (h :: h' :: l) = sortedb (h' :: l)`.

Proof. (`*` `....` `*`) **Qed.**

Lemma `sortedb_cons_cons_nleb` (`h h' : nat`) (`l : list nat`) :
`h <= h' = false -> sortedb (h :: h' :: l) = false`.

Proof. (`*` `....` `*`) **Qed.**

3. Donner la preuve en Rocq de

Lemma `sortedb_sorted` (`l : list nat`) : `sortedb l = true -> sorted l`.

4. Donner la preuve en Rocq de

Lemma `sorted_sortedb` (`l : list nat`) : `sorted l -> sortedb l = true`.

5. En déduire :

Lemma `sorted_decidable` (`l : list nat`) : `sorted l \ / ~ sorted l`.

--- * ---